



Tecnológico de Monterrey

Campus Santa Fe

Nombre de la actividad:

Actividad en clase - Ante-proyecto bajo los principios del ML Ops

Actividad en clase - Riesgos, decisiones, checks y gates

Actividad en clase - Despliegue, Monitoreo y Testing

Nombre del curso:

Inteligencia artificial avanzada para la ciencia de datos II (Gpo 101)

Alumnos:

Andrés Jaramillo Barón - A01029079

Pedro Mauri Mtz - A01029143

Profesor:

Edoardo Bucheli Susarrey

Fecha de entrega:

27/11/2025

Descripción de proyecto:

Herramienta capaz de predecir valores de activos financieros, utilizando datos históricos en conjunto con un análisis de sentimiento de noticias acerca de los mismos stocks; es decir, también revisa noticias que afecte al activo analizado, con el objetivo de ayudar a tomar la decisión adecuada con el análisis.

Ante-proyecto bajo los principios del ML Ops

Resumen ejecutivo:

1.1 Elevator Pitch

Nuestro proyecto desarrolla una plataforma inteligente para analizar y anticipar el comportamiento de activos financieros, integrando precios, noticias y sentimiento de mercado en un mismo flujo de decisión. El sistema combina señales cuantitativas (series temporales de precios, volumen y volatilidad) con señales cualitativas (texto de noticias y su sentimiento), de modo que el usuario pueda interpretar tanto el “qué pasó” como el “por qué podría pasar”. A través de un pipeline automatizado y alineado con principios de MLOps, buscamos que la plataforma sea confiable, escalable, adaptable y mantenible: confiable por su validación y trazabilidad de datos/modelos; escalable por su capacidad de procesar múltiples activos y fuentes; adaptable por su retraining y su diseño desacoplado; y mantenible por su versionamiento, monitoreo continuo y despliegues controlados. En conjunto, la plataforma transforma grandes volúmenes de información financiera en predicciones accionables para respaldar decisiones más rápidas y basadas en evidencia.

1.2 Alcance de la solución

La plataforma se apoya en dos componentes principales: un modelo de predicción de precios/retornos a partir de series temporales, y un modelo de análisis de sentimiento aplicado a noticias financieras vinculadas a los mismos activos. El valor del sistema no se limita a entrenar modelos aislados, sino a sostener un ciclo completo de datos -> features -> evaluación -> inferencia -> monitoreo, manteniendo coherencia entre entrenamiento y producción.

Enfoque MLOps, arquitectura y decisiones base:

2.1 Principios MLOps como criterios de diseño

Este documento adopta como marco principal *Designing Machine Learning Systems* (Huyen, 2022). Por eso, las decisiones de datos, evaluación, serving, monitoreo y despliegue se presentan como partes de un mismo sistema, no como pasos aislados. A continuación revisaremos los principios de MLOps.

2.1.1 Confiable

La confiabilidad se traduce en predicciones verificables y en evidencia reproducible. Para ello, cada etapa del pipeline incorpora validaciones automáticas de calidad de datos, control de versiones y experiment tracking, asegurando que los modelos se entrenen con información consistente y que los resultados puedan repetirse. En paralelo, se evalúa continuamente el desempeño con métricas apropiadas al riesgo del dominio financiero, de modo que el sistema pueda detectar degradaciones cuando la dinámica del mercado cambie.

2.1.2 Escalable

La escalabilidad aparece como capacidad de crecer en número de activos, fuentes y volumen de datos sin degradar el rendimiento. Esto exige un diseño modular por etapas (ingesta, transformación, almacenamiento, entrenamiento, serving) y un modelo de datos que permita consultas eficientes. La meta es soportar más activos y más señales simultáneas manteniendo un pipeline estable y operativo.

2.1.3 Adaptable

La adaptabilidad apunta a sobrevivir cambios estructurales del mercado y del lenguaje en noticias. Por eso se contempla retraining programado y evaluación fresca sobre ventanas recientes, junto con una arquitectura desacoplada que permita reemplazar componentes sin rehacer todo el sistema. La intención es que el proyecto pueda migrar de enfoques estadísticos a modelos más complejos cuando el volumen de datos y la necesidad de precisión lo justifiquen, manteniendo el mismo flujo general.

2.1.4 Mantenible

La mantenibilidad se logra con prácticas de ingeniería: versionamiento de datasets/modelos/configuración, documentación clara, pipelines automatizados y monitoreo

de métricas y logs. Esto permite aislar fallas, reproducir experimentos, ejecutar rollbacks y evolucionar el sistema sin interrumpir su operación.

2.2 Conceptos teóricos y enfoque de modelado

El proyecto se apoya en fundamentos de Machine Learning y Ciencia de Datos: representación estructurada, extracción de features, validación y evaluación rigurosa, y retroalimentación para mejorar desempeño. En la parte de series temporales se consideran enfoques clásicos (regresión, árboles, random forest, gradient boosting, SVM) y modelos orientados a secuencias (LSTM, GRU). Para texto y sentimiento, se contemplan embeddings y enfoques basados en Transformers, priorizando el equilibrio entre precisión, interpretabilidad, costo computacional y datos disponibles. En esta etapa del reporte, el objetivo es justificar el marco de decisión; la selección final se hará con evidencia de baselines y evaluación offline.

2.3 Tecnologías e infraestructura

La implementación se desarrollará principalmente en Python por su ecosistema para pipelines y ML, usando librerías como pandas, NumPy y scikit-learn, además de herramientas de extracción automatizada para mercado y noticias (por ejemplo, yfinance y utilidades para RSS y scraping selectivo). Para modelos más avanzados se evaluará PyTorch o TensorFlow. La interfaz se plantea con TypeScript y React para visualizar datos, tendencias y predicciones en un dashboard. El almacenamiento se basa en PostgreSQL, con posibilidad de extender hacia un motor de series temporales en fases posteriores. Para replicabilidad y aislamiento se utilizará Docker; el control de versiones se gestionará con Git/GitHub y la automatización de pruebas y tareas con GitHub Actions, dejando la arquitectura preparada para migrar a monitoreo y orquestación más avanzada conforme el sistema madure.

2.4 Análisis sobre datos (visión general)

Este apartado resume cómo se obtienen, estructuran y controlan los datos del proyecto para que el sistema sea reproducible y coherente a lo largo del ciclo completo (ingesta -> almacenamiento -> entrenamiento -> serving). Al tratarse de una plataforma que combina precios y noticias, el diseño prioriza trazabilidad, consistencia temporal y calidad, ya que cualquier degradación en datos se refleja directamente en el desempeño del modelo.

2.4.1 Fuentes y formatos (cuantitativo + cualitativo)

El sistema integra fuentes heterogéneas: APIs financieras para señales cuantitativas (precios, volumen, variaciones) y feeds RSS junto con scraping selectivo para señales cualitativas (noticias, titulares y contexto). Los datos de mercado se manejan en estructura tabular para facilitar análisis y construcción de features, mientras que los textos se almacenan en formatos tipo JSON para preservar metadatos como fecha, fuente y entidad, y permitir trazabilidad.

2.4.2 Modelo de datos y almacenamiento

La información se organiza bajo un modelo relacional que conecta activos, precios, noticias y sus relaciones, manteniendo integridad referencial y evitando duplicidades. El almacenamiento se centraliza en PostgreSQL por su estabilidad y soporte para consultas analíticas, con la posibilidad de extenderse hacia un motor orientado a series temporales en fases posteriores si el volumen y la frecuencia lo justifican.

2.4.3 Pipelines (extracción, limpieza, enriquecimiento)

El procesamiento se implementa como un pipeline automatizado que cubre extracción, validación, limpieza, enriquecimiento y persistencia. Su ejecución mediante scripts en Python y contenedores Docker ayuda a mantener reproducibilidad y aislamiento de entornos. La modularidad del pipeline permite que el sistema crezca sin rehacerlo completo, incorporando nuevas fuentes o etapas conforme evoluciona el proyecto.

2.4.4 Sampling, etiquetado y riesgos temporales

Para entrenamiento se contempla weighted sampling, buscando dar mayor peso a eventos críticos en mercado y compensar desbalances en clases de sentimiento, sin romper la consistencia temporal. Este enfoque exige controles explícitos para evitar data leakage, especialmente cuando la información textual puede estar desfasada respecto al movimiento del precio. Por ello, se asume un split temporal estricto y evaluación temporal para conservar causalidad y evitar métricas infladas offline.

2.4.5 Checks y gates de calidad

Antes de entrar al pipeline, los datos pasan por validaciones estructurales (nulos, duplicados, formatos, rangos y consistencia temporal) y por normalización entre fuentes (fechas, zonas horarias, unidades y nombres). En noticias se agregan controles semánticos para asegurar que

el texto esté relacionado con el activo correcto. Cada lote se audita y versiona para mantener trazabilidad y permitir reproducibilidad de experimentos.

Riesgos, decisiones, checks y gates

Fuentes de datos y diseño del modelo de datos

(Capítulo 3):

3.1 Fuentes de datos (ingesta automatizada e híbrida)

Es importante pensar en la automatización de la obtención de datos: si dependemos de descargas manuales, rompemos reproducibilidad y escalabilidad. Por eso planteamos una estrategia de ingesta **híbrida**. Para activos financieros se priorizan **APIs oficiales** o librerías confiables (p. ej., *yfinance*, Alpha Vantage, CCXT) que entregan datos históricos y, cuando aplique, señales cuasi en tiempo real de forma estructurada. Para noticias, la mejor práctica es combinar **RSS + APIs de agregadores** (p. ej., NewsAPI, GDELT) y usar **scraping selectivo** como respaldo cuando no exista otro acceso. Con esto buscamos un pipeline **estable, trazable y mantenible**.

3.2 Riesgos de las fuentes y mecanismos de control

También es crítico entender los riesgos inherentes a cada fuente: los RSS suelen tener ventanas limitadas de artículos, las APIs imponen *rate limits* y coberturas distintas, y el scraping es frágil ante cambios en HTML y restricciones legales. Para mitigar, el diseño incorpora **capas de fallback**, **deduplicación** y **normalización por entidad** (ticker, ISIN, sinónimos) para mapear cada noticia al activo financiero correspondiente. Esto sigue la premisa del libro: el sistema de ML no es sólo el modelo, sino un **ecosistema de datos + ingesta + validación** que debe poder evolucionar sin romperse al escalar.

3.3 Formato / modelo de datos (esquema relacional)

El almacenamiento se plantea como un modelo **relacional** con entidades para activos financieros, precios históricos, noticias y sus relaciones. Esto permite mantener integridad referencial, evitar duplicidades y habilitar consultas complejas que mezclen precios + noticias por activo y ventana temporal. Además, el modelo es extensible: si se integran nuevas fuentes

(redes sociales, reportes financieros), se añaden tablas conectadas al esquema existente sin rediseñar todo.

3.4 Consideraciones de procesamiento, espacio, disponibilidad y pipeline

En procesamiento y almacenamiento se prioriza eficiencia, disponibilidad y mantenibilidad. Como los datos provienen de múltiples fuentes (APIs, RSS y scraping selectivo), se implementan procesos de ingesta automatizada y limpieza periódica para asegurar consistencia temporal y calidad. Siguiendo *Designing Machine Learning Systems*, el pipeline se diseña de forma **modular**, separando obtención, transformación y persistencia para escalar conforme crece el volumen.

En cuanto a espacio y disponibilidad, se combinan formatos ligeros y comprimidos para históricos con una base relacional (p. ej., PostgreSQL) para acceso rápido y consultas analíticas. Se definen políticas de retención y respaldos que balancean costo y disponibilidad: datos recientes en almacenamiento “caliente” y históricos en “frío”. Además, se contempla tolerancia a fallos mediante backups automáticos, versionado de datasets y alertas ante caídas o inconsistencias, manteniendo un sistema confiable y reproducible a largo plazo.

Riesgos y decisiones clave del training dataset

(Capítulo 4):

4.1 Sampling (por modelo y riesgos asociados)

Para el modelo de activos financieros usaremos **weighted sampling** para dar más relevancia a eventos críticos (crashes, anuncios macro, crisis corporativas) sin romper la secuencia temporal. Los riesgos incluyen: *prior shift* hacia regímenes extremos por sobre-ponderación; *look-ahead bias/data leakage* si los pesos se definen con información posterior al horizonte; reducción del tamaño muestral efectivo por autocorrelación en ventanas contiguas; sobreajuste por régimen y pérdida de generalización en periodos tranquilos; descalibración en baja volatilidad; y validación sesgada si no se emplean splits temporales.

Para el modelo de sentimiento (NLP) también preferimos **weighted sampling** para dar más presencia a clases menos frecuentes sin forzar un balance artificial. Riesgos: distribución de sentimientos poco realista si se sobre-ponderan clases; amplificación de ruido de etiquetas cuando instancias mal clasificadas reciben más peso; correlación por sindicación/duplicados;

leakage por *entity linking* mal alineado; desalineación temporal que invierte causalidad (noticia posterior al movimiento); y drift de dominio entre fuentes/épocas que el re-ponderado puede exacerbar.

4.2 Data labeling (supervisión y costo de calidad)

Como el enfoque es supervisado, debemos decidir el método para definir clases y etiquetas. Se plantea **hand-labeling** con apoyo de una estudiante de finanzas (ITESM CSF). Ejemplos: dirección del movimiento (sube/baja/mantiene), magnitud del cambio (porcentaje) y sentimiento (positivo/negativo/neutral). Reconocemos que el hand-labeling es costoso: requiere análisis cuidadoso para asegurar calidad y, en producción real, varios especialistas; con una sola etiquetadora, esta fase probablemente será lenta y deberá administrarse con subsets bien definidos.

4.3 Class imbalance (impacto y estrategias)

Los modelos suelen entrenar mejor cuando no hay desbalances severos. En finanzas, eventos extremos y noticias negativas suelen ser minoría, pero son casos críticos: si el modelo falla ahí, el costo es alto. Por eso se usa weighted sampling. Como alternativa, se puede aplicar **oversampling** (síntesis de datos mediante combinaciones convexas) con cuidado de no introducir artefactos. En paralelo, se deben definir métricas objetivo acordes al problema para no optimizar únicamente promedios que escondan fallas en minorías.

4.4 Data augmentation (robustez, especialmente en NLP)

El data augmentation permite generar nuevas muestras a partir de las existentes. Métodos típicos: transformaciones que preservan etiqueta, perturbación (agregar ruido) y síntesis. El libro menciona que ha ganado terreno en NLP por la facilidad de usar sinónimos/paráfrasis, por lo que resulta especialmente útil para fortalecer el modelo de sentimiento y mejorar robustez ante variaciones superficiales del texto.

4.5 Train / Validation / Test split (consistencia temporal)

Tras recolectar y limpiar los datasets, se define el split de entrenamiento/validación/prueba. Una proporción razonable puede ser **70/15/15**, pero en series de tiempo debe respetarse el orden temporal para evitar leakage. Además, es deseable complementar con evaluación temporal (p. ej., rolling/expanding) para verificar estabilidad del desempeño a través de distintos regímenes de mercado.

4.6 Data leakage (riesgo central del dominio)

Un riesgo alto es que parte de la información no esté disponible en el momento real de decisión: noticias pueden publicarse después del movimiento que se intenta predecir. Por ello, se valida que las secuencias sean temporalmente consistentes y que no se utilice información del “futuro”. Este control es crítico porque el leakage puede inflar métricas offline y colapsar desempeño en producción.

4.7 Privacy and ethical data collection

Aunque precios y noticias suelen ser públicos, se debe cuidar no incorporar información personal, respetar licencias/derechos de autor y limitar la recolección a lo necesario para el objetivo del modelo. Esto reduce riesgos legales y asegura una práctica responsable de ingeniería de datos.

4.8 Checks y gates de calidad de datos (del entrenamiento a producción)

Antes de que los datos entren al pipeline de entrenamiento o análisis, pasan por checks estructurales: nulos, duplicados, outliers, rangos inválidos y formatos incorrectos (p. ej., fechas mal codificadas, precios negativos). En datos financieros se valida consistencia cronológica y ausencia de rupturas; en texto se exige fecha, fuente y cuerpo válido. Cualquier anomalía activa un **gate** que bloquea el avance del lote hasta corregir o descartar.

Dado que la ingesta proviene de APIs, RSS y scraping selectivo, se aplican gates de **normalización** (variables, zonas horarias, idioma, unidades) y controles de duplicación y consistencia cruzada, conservando la versión más completa o confiable según la jerarquía definida. Además, se ejecutan checks **semánticos**: *entity linking* para filtrar noticias irrelevantes y reducir ruido, y validación de variaciones extremas de mercado contra patrones/eventos para detectar errores de captura.

Finalmente, se incluyen gates de **auditoría y versionado**: origen, fecha de ingesta, resultados de validación y un identificador de versión (hash). Esto asegura trazabilidad end-to-end, permite reproducir experimentos y mantiene alineación con buenas prácticas de MLOps a lo largo del ciclo de vida del sistema.

Despliegue, Monitoreo y Testing.

Evaluación Offline (Capítulo 6):

En esta sección definimos cómo evaluaremos nuestros modelos antes de pensar en desplegarlos: (1) el modelo de series de tiempo para precios de activos y (2) el modelo de NLP para sentimiento en noticias financieras. Siguiendo el enfoque de *offline model evaluation* del libro, empezamos comparando contra líneas base simples y, después, aplicamos una batería de pruebas de robustez, calibración y análisis por segmentos (slices).

6.1 Baselines para comparar el modelo

Antes de decir que “el modelo funciona”, necesitamos saber contra qué lo comparamos. Todas las baselines se correrán con el mismo split temporal, las mismas ventanas y los mismos activos, usando las mismas métricas y registrándose en un sistema de *experiment tracking* para mantener resultados reproducibles.

6.1.1 Regla cero / naive (persistencia y promedio)

Para precios, la baseline más simple es la persistencia: asumir que el mejor estimado de mañana es el precio de hoy (o que el retorno esperado es aproximadamente cero). Esto es equivalente al *zero rule*: una heurística mínima que cualquier modelo serio debería superar con claridad. En el caso del sentimiento, la versión naive consiste en predecir siempre la clase mayoritaria, típicamente “neutral”, que suele dominar en noticias financieras.

Como segundo escalón, usamos una media móvil de retornos: el retorno futuro se aproxima al promedio de los retornos recientes en una ventana corta (por ejemplo, 5 o 20 días). Estas baselines definen el “mínimo aceptable”: si el modelo complejo no mejora de forma consistente, es señal de features débiles, fuga de información o problemas de implementación.

6.1.2 Modelo aleatorio estratificado por volatilidad

Además del naive, incluimos una referencia de “azar con estructura”. En mercados financieros el comportamiento no es uniforme; depende fuertemente de la volatilidad. Por eso, segmentamos el histórico de cada activo en buckets de volatilidad (baja, media, alta) usando, por ejemplo, desvío estándar de retornos en ventanas móviles. En cada bucket

estimamos la distribución empírica de movimientos (sube/baja/se mantiene) y, al predecir, identificamos el bucket actual y sampleamos una dirección siguiendo esa distribución.

Este baseline sirve para cuantificar un “lift” realista sobre el azar: si el modelo apenas iguala o mejora marginalmente este muestreo estratificado, significa que está capturando poca señal y que su ventaja sobre heurísticas simples sigue siendo limitada.

6.1.3 Solución existente (ARIMA/Prophet/XGBoost base)

El libro recomienda comparar también contra lo “razonable” que ya usaría un equipo sin complicarse demasiado. Para precios, consideramos como baseline un modelo clásico por activo (ARIMA o Prophet) y, alternativamente, un modelo tabular tipo XGBoost con features estándar como retornos rezagados, volatilidad reciente y volumen. El enfoque final del proyecto sólo se considerará exitoso si supera de forma consistente esta solución existente, no sólo a reglas naive.

6.1.4 Baseline humano (si aplica)

Cuando sea posible, añadimos una referencia humana como techo aproximado. Un perfil con formación financiera puede etiquetar un subconjunto de noticias y ventanas de mercado, indicando dirección esperada del activo (sube/baja/mantiene) y sentimiento (positivo/negativo/neutral). Esto ayuda a decidir dónde conviene invertir esfuerzo: si estamos claramente por debajo, hay margen de mejora; si estamos cerca, puede ser más valioso priorizar robustez, explicabilidad o latencia.

6.2 Métodos de evaluación: robustez, invariancia y análisis por segmentos

La evaluación offline no se reduce a un único número. Para evitar depender de un solo número, estresamos el modelo con: splits temporales, perturbaciones controladas, invariancias deseables, análisis por slices y calibración de probabilidades.

6.2.1 Split temporal y backtesting (rolling/expanding)

Para evitar *data leakage*, respetamos el orden temporal. Usaremos un split estático por periodos (train/valid/test en bloques cronológicos) y lo complementaremos con backtesting usando ventanas *rolling* (tamaño fijo) y *expanding* (el entrenamiento crece con el tiempo). Con esto verificamos que el modelo no “funciona” sólo en un tramo afortunado del histórico, sino que mantiene desempeño razonable a través de distintos regímenes de mercado.

6.2.2 Pruebas de perturbación (perturbation tests)

Aplicamos cambios pequeños a inputs que, idealmente, no deberían alterar materialmente la predicción. En precios esto incluye ruido leve, errores de redondeo o latencia, y jitter en timestamps dentro de la misma granularidad. En noticias esto incluye variaciones superficiales como espacios extra, acentos, mayúsculas/minúsculas, reordenamientos menores o sustitución por sinónimos que no cambian el significado. Si el desempeño se degrada de forma exagerada ante perturbaciones mínimas, lo tratamos como una señal de fragilidad o sobreajuste.

6.2.3 Pruebas de invariancia (invariance tests)

Además de perturbaciones, chequeamos transformaciones que preservan la etiqueta. Por ejemplo, si el modelo trabaja sobre retornos, escalar los precios por un factor constante no debería cambiar la predicción de dirección; normalizar timestamps a UTC manteniendo el orden relativo tampoco debería alterar resultados; y normalizaciones de texto (espacios, comillas, mayúsculas) no deberían modificar el sentimiento. Una sensibilidad alta a estas transformaciones suele indicar problemas de preprocesamiento o de diseño de features.

6.2.4 Evaluación por slices (slice-based evaluation)

Los promedios esconden fallas graves. Por eso evaluamos por segmentos: régimen de mercado (alta vs baja volatilidad, crisis vs periodos tranquilos), tipo de activo (sector, tamaño, ETFs), horizonte (intradía/diario/semanal) y fuente/horario de noticias (apertura/cierre vs sesión intermedia). Reportamos métricas por slice (MAPE/MSE/MAE, directional accuracy y F1 por clase). Si detectamos slices con caídas fuertes, se documentan como riesgos conocidos y se convierten en insumo directo para el plan de mejoras.

6.2.5 Calibración, métricas y señales operativas

En precios y retornos usamos métricas de error numérico como MAPE y MAE/MSE, y complementamos con métricas de dirección como *directional accuracy* y matrices de confusión por slice. Si el modelo devuelve probabilidades, revisamos calibración con *reliability plots* y Brier score, buscando que las probabilidades correspondan a frecuencias reales (por ejemplo, “70% sube” implique que sube ~7 de cada 10). Aunque sea offline, también medimos señales operativas en simulación, como latencia por muestra o batch y throughput, anticipando escenarios batch y online.

En conjunto, estas pruebas nos permiten confiar en que el desempeño offline no proviene de azar o fuga de información, sino de patrones relativamente estables en mercado y noticias, y sirven como puente directo hacia despliegue, monitoreo y manejo de riesgos del sistema completo.

Tipos de inferencia y pipeline (Capítulo 7):

En este capítulo describimos cómo se usaría el modelo en la práctica a lo largo de la vida del proyecto: desde experimentos iniciales, pasando por un piloto con usuarios internos, hasta un despliegue en producción. Siguiendo el enfoque del libro, no existe un único tipo de inferencia “correcto”; el sistema evoluciona de **batch** -> **híbrido** -> **online** conforme aumentan los requisitos de latencia, estabilidad y mantenimiento.

El proyecto combina datos históricos de mercado (precios, volumen, volatilidad) con sentimiento de noticias sobre los mismos activos. Por eso, el pipeline debe orquestar features históricos (medias móviles, retornos) junto con señales casi en tiempo real (sentimiento reciente), manteniendo consistencia entre lo que se construye en entrenamiento y lo que se usa para servir predicciones.

7.1 Visión general del pipeline

A nivel alto, el sistema se divide en dos flujos complementarios. El primero es el de entrenamiento offline (*training pipeline*), donde se ingieren precios y noticias, se limpian y alinean temporalmente, se construyen features y se entrena/evalúa el modelo siguiendo la metodología del capítulo 6. El resultado de este flujo es una versión registrada del modelo con sus métricas, hiperparámetros y artefactos asociados, de modo que el equipo pueda rastrear exactamente qué se entrenó y con qué datos.

El segundo flujo es el de inferencia (*serving pipeline*). Aquí el sistema calcula features “al vuelo” o mediante jobs programados, carga la versión aprobada del modelo y emite predicciones (retorno esperado y/o dirección) para consumo del usuario. Lo que cambia según la fase del proyecto no es la idea del pipeline, sino **cuándo** y **cómo** se ejecuta esta inferencia.

7.2 Fase exploratoria / research: inferencia batch

En la etapa inicial priorizamos iterar rápido y barato, sin exigencias de milisegundos. Por eso

empezamos con *batch prediction* sobre datos históricos. En la práctica, esto implica construir un *feature store* histórico con tablas por activo y timestamp que incluyan retornos rezagados, volatilidad, volumen, indicadores técnicos y agregaciones de noticias por ventana (por ejemplo, sentimiento promedio y conteo de titulares positivos/negativos).

Sobre ese *feature store* corren jobs periódicos, típicamente nocturnos, que congelan una versión del dataset, entrenan candidatos, generan predicciones sobre rangos backtestables y guardan salidas para análisis. La evaluación se realiza offline con métricas como MAPE y directional accuracy, y se registra todo en un sistema de *experiment tracking* para preservar reproducibilidad. En esta fase no hace falta un servicio online: basta con scripts o notebooks que lean features y escriban resultados en tablas o archivos.

7.3 Fase piloto interno: pipeline híbrido

Con modelos prometedores, el objetivo pasa a ser “usable” sin asumir todavía complejidad de producción. Aquí encaja un esquema híbrido: parte batch y parte online ligera. El componente batch recalcula features recientes en un intervalo definido (hora/día), ejecuta el modelo aprobado y escribe predicciones a una tabla de “predicciones vigentes”. El componente online, en lugar de ejecutar el modelo, expone una API o dashboard que simplemente consulta esas predicciones y permite filtrar por activo, sector o ventana temporal.

Este diseño da una experiencia casi en tiempo real con costos operativos bajos. También hace sencillo el rollback, porque cambiar de versión es cuestión de corregir el job o apuntar a otra tabla/modelo registrado. En paralelo, se incorporan las primeras piezas de monitoreo básico, como salud de jobs, errores en features y latencias de ingesta, todavía sin SLOs estrictos.

7.4 Fase de producción: online con soporte batch

Si el caso de uso exige decisiones más reactivas (minutos o segundos), la inferencia pasa a ser mayormente online, y el batch queda como soporte para reentrenamiento, re-puntuación histórica y análisis. En este esquema el modelo se despliega como servicio (REST/gRPC) que recibe el identificador del activo y features recientes (o referencias para recuperarlas desde un feature store online) y devuelve probabilidad de subida/bajada, retorno esperado y, opcionalmente, explicaciones simples.

Para soportar esto, la ingesta de precios y noticias se acerca a un modo streaming o cuasi real (colas/streams), y el cómputo de features debe resolverse con ventanas deslizantes y

agregaciones recientes de sentimiento. A la par, se mantiene un pipeline batch periódico para reentrenar y reevaluar offline; si el modelo cumple criterios, se promueve con una estrategia de despliegue segura. En esta fase la observabilidad se vuelve central: latencia, disponibilidad, drift y errores dejan de ser señales “nice-to-have” y pasan a ser condiciones de operación.

7.5 Unificación de batch y streaming (evitar train-serve skew)

El libro advierte que uno de los problemas clásicos es terminar con dos mundos: uno para entrenamiento batch y otro para serving online, con lógicas duplicadas para construir features. Eso crea *train-serve skew* y hace que el modelo en producción “vea” representaciones distintas a las del entrenamiento. Para reducir este riesgo, el diseño busca compartir código de features en librerías únicas, de modo que batch y streaming llamen a las mismas funciones para retornos, indicadores y agregaciones de sentimiento.

En la misma dirección, se plantea un feature store unificado que soporte lecturas batch para entrenamiento/backtesting y lecturas online de baja latencia para serving. Finalmente, se incorporan checks automáticos que comparen distribuciones offline vs online y disparen alertas cuando un feature salga fuera de rango o muestre señales de desalineación. Con esta unificación, pasar de batch a online deja de ser un “rework” y se vuelve una evolución natural del mismo stack.

Proceso de monitoreo (Capítulo 8):

En este capítulo describimos los riesgos principales al llevar el modelo a producción y cómo se conectan con un monitoreo continuo. Siguiendo la taxonomía del libro, nos enfocamos en tres fallas especialmente relevantes para este proyecto: (1) datos de producción distintos a entrenamiento, (2) edge cases, y (3) degenerate feedback loops. A partir de estos riesgos, definimos un proceso de monitoreo que cubre predicciones, features y raw inputs, alineado con los principios de MLOps del proyecto.

8.1 Datos diferentes entre entrenamiento y producción

Una causa frecuente de fallas es el *distribution shift*: en producción, los datos dejan de parecerse a los de entrenamiento (covariate shift, concept drift, label shift, etc.). En un contexto financiero este riesgo es alto porque los mercados cambian de régimen (crisis, baja volatilidad, rallies) y esas transiciones alteran la relación entre features y objetivo. A esto se

suma el drift de dominio en noticias, donde cambian estilo, vocabulario o frecuencia, y la dependencia de múltiples fuentes (APIs, RSS, scraping selectivo) que pueden modificar formato, cobertura o calidad con el tiempo.

En producción, este problema suele manifestarse como caídas bruscas de desempeño después de eventos macro o cambios estructurales, o como degradación concentrada en ciertos slices (alta volatilidad, sectores específicos, activos particulares). También puede verse en desalineaciones claras de distribuciones de features, como volatilidad fuera de rango o cambios notables en el vocabulario de noticias.

Para mitigarlo, el monitoreo debe operar en dos niveles. Primero, se monitorea desempeño en cuanto haya etiquetas disponibles, con métricas como MAPE/MAE/MSE, directional accuracy y F1 en sentimiento. Segundo, se monitorean distribuciones de features mediante estadísticos simples y tests de drift como KS o PSI. Además, se complementa con backtesting “fresco” en ventanas recientes y con retraining programado (semanal/mensual), priorizando datos recientes sin sobreajustar a episodios extremos.

8.2 Edge cases

Los edge cases son raros en el histórico, pero su impacto puede ser desproporcionado en producción. En este proyecto el riesgo es medio–alto porque eventos como crashes, splits, suspensiones o datos inconsistentes no aparecen con frecuencia, pero suelen coincidir con momentos críticos para decisiones. En noticias también aparecen casos difíciles, como titulares irónicos, sarcásticos o cambios abruptos de tono, y la heterogeneidad de fuentes puede introducir picos, gaps o “tirones” de datos.

Estos casos suelen manifestarse como predicciones absurdas durante flash crashes o rallies extremos, señales incorrectas por splits o gaps mal interpretados, o clasificación errónea de noticias claramente negativas como neutrales o positivas justo cuando el tono del mercado cambia.

Para reducir su impacto, se combinan defensas que actúan antes y después de la predicción. En los raw inputs se implementa detección de outliers (precio/volumen, gaps) y validaciones de consistencia. En la capa de decisión se agregan guardrails que bloqueen acciones automáticas bajo volatilidad/spread extremos o que activen checks adicionales cuando la señal cambia demasiado respecto a periodos anteriores. En paralelo, se aumenta la cobertura de extremos mediante weighted sampling y/o augmentation, evaluando explícitamente slices

de crisis, y se mantiene logging detallado con revisiones post-mortem para ajustar modelo y reglas conforme aparezcan nuevos casos.

8.3 Degenerate feedback loops

El modelo puede distorsionar los datos futuros si influye en qué se observa o decide, terminando en bucles que refuerzan sesgos. En este proyecto el riesgo es medio: si la interfaz prioriza activos o noticias según el modelo, cambia qué se consulta y, con el tiempo, qué datos “existen” en la práctica; y si las salidas alimentan filtros posteriores, se reduce diversidad y el sistema se vuelve ciego a señales no favorecidas.

En producción, esto suele verse como auto-confirmación, donde el modelo termina “viendo” sólo lo que ya prefiere. Algunos activos se vuelven invisibles por baja prioridad y el dataset se sesga hacia un subconjunto de señales. El síntoma más engañoso es que las métricas parecen buenas dentro del subconjunto observado, mientras el desempeño global del sistema empeora.

La mitigación requiere preservar exploración y medir influencia. Se introduce randomización controlada para mantener parte del contenido no priorizado por el modelo, se registran exposiciones y acciones (qué vio el usuario y qué hizo) y se incorporan señales exógenas que no dependan del propio modelo. Además, se usan experimentos controlados (A/B o shadow) para comparar variantes y detectar si ciertos sesgos se amplifican con el tiempo.

8.4 Proceso de monitoreo en producción

El monitoreo se organiza en tres capas: predicciones, features y raw inputs, soportadas por dashboards y alertas accionables. La idea es detectar degradación temprano, localizar si el problema viene del modelo o de los datos, y reaccionar con procedimientos claros.

8.4.1 Predicciones y desempeño

En la capa de salidas del modelo se monitorean métricas cuando existan etiquetas, como MAPE/MAE/MSE, directional accuracy y F1/accuracy por clase en sentimiento. Si se predicen probabilidades, se revisa calibración con Brier score y reliability plots en ventanas recientes. Además, el desempeño se desglosa por slices —activo, sector, régimen de volatilidad, fuente y horizonte— y se incluyen percentiles (p50/p90/p95) para capturar colas de error que suelen ocultarse en promedios.

8.4.2 Features y raw inputs

En raw inputs se monitorea la latencia de feeds, el porcentaje de campos críticos faltantes, y la frecuencia/tamaño de lotes. En features se monitorean estadísticos contra entrenamiento, indicadores de drift (KS/PSI), staleness de agregaciones (por ejemplo, sentimiento de la última hora) y checks de train–serve skew para asegurar consistencia entre la lógica usada en entrenamiento y producción. Este monitoreo se integra con validaciones ya definidas para el dataset (consistencia temporal, auditoría y versionado), extendiéndolas al entorno productivo.

8.4.3 Alertas operables y operación

El monitoreo debe traducirse en alertas operables, no sólo en gráficos. Se disparan alertas por degradación del modelo cuando sube el p95 del error, caen bruscamente directional accuracy o F1, o aparece descalibración fuerte. Se disparan alertas por drift/datos cuando cambian significativamente features clave o aumentan missing values, duplicados o inconsistencias. Y se incluyen alertas de sistema cuando la latencia supera el SLO, aparecen timeouts, caen servicios o falla la base de datos.

Cada alerta debe apuntar a un playbook: qué dashboard revisar, cómo distinguir un bug del pipeline frente a un cambio real de mercado o un incidente de infraestructura, y cómo documentar la acción correctiva para mantener trazabilidad del sistema.

Metodología y tests de producción (Capítulo 9):

En este capítulo describimos cómo introducir nuevos modelos en producción sin romper el sistema ni degradar la experiencia del usuario. Siguiendo el libro, cada cambio se trata como un experimento controlado: se comparan variantes, se monitorean métricas y se definen criterios claros de *go/no-go*. En nuestro proyecto (predicción de activos + sentimiento), esto implica probar versiones nuevas, compararlas contra el modelo actual y contra baselines, y desplegarlas de manera gradual con capacidad de rollback.

9.1 Objetivos de los tests en producción

Los tests en producción buscan cuatro objetivos que guían el diseño de los experimentos. Primero, seguridad: evitar degradaciones fuertes sin detección temprana. Segundo, evidencia: tomar decisiones con métricas y no con intuiciones. Tercero, trazabilidad: saber qué versión corrió, con qué configuración y en qué periodo. Cuarto, iteración controlada: introducir mejoras de forma gradual entendiendo su impacto real dentro del sistema. Las técnicas de

este capítulo son distintas combinaciones de estos objetivos, con diferentes niveles de riesgo y complejidad operativa.

9.2 Shadow deployment

El *shadow deployment* (o *dark launch*) suele ser el primer paso porque es el más seguro. El modelo nuevo recibe exactamente las mismas solicitudes que el modelo actual, pero sus salidas no afectan lo que ve el usuario. En paralelo, las predicciones se registran y se comparan offline contra el modelo en producción, las etiquetas que van llegando y las métricas definidas en capítulos anteriores.

En el proyecto esto se aplica manteniendo el servicio de inferencia con el modelo A, mientras que las mismas features se envían al modelo B como candidato. El usuario sigue viendo únicamente A, y B queda registrado en logs o tablas internas. La comparación periódica A vs B se hace con métricas como MAPE/MSE, directional accuracy, F1 de sentimiento y desempeño por slices. La ventaja central es que hay cero riesgo directo para el usuario y se pueden detectar bugs, drift o latencias antes de exponer el modelo. La limitación es que todavía no se observa el impacto en comportamiento del usuario y se requiere capacidad extra de cómputo y almacenamiento de logs.

9.3 A/B testing

En el A/B testing se divide tráfico en grupos para comparar variantes en producción con KPIs definidos. La unidad de asignación puede ser usuario (para consistencia), activo (para una comparación controlada) o sesión (más simple de implementar). En este proyecto, además de métricas técnicas como MAPE/MSE por slice, directional accuracy y latencia media/p95, también pueden considerarse métricas de uso si existe interacción humana, como consultas repetidas sobre un activo, tiempo de lectura o feedback.

Operativamente, el A/B requiere definir duración mínima y tamaño de muestra, de modo que el experimento tenga potencia suficiente. También debe existir un plan de rollback si el modelo B empeora de forma clara. A diferencia del shadow, el A/B sí modifica lo que ve el usuario, por lo que se usa cuando el modelo ya mostró estabilidad básica en pruebas previas.

9.4 Canary testing

El canary testing despliega el modelo nuevo a una fracción pequeña del tráfico (por ejemplo 1–5%) y aumenta exposición si las métricas se mantienen dentro de rangos aceptables. El flujo típico empieza con ese porcentaje reducido, monitoreando en tiempo casi real error,

directional accuracy, latencia y tasa de fallas del servicio. Si todo se ve bien, se escala a porcentajes intermedios (por ejemplo 20–30%) y finalmente al 100%. Si en algún punto se cruzan umbrales críticos, se ejecuta rollback, idealmente automático.

En el proyecto, el canary tiene sentido después de shadow (y quizá un A/B limitado) porque protege contra fallas reales no vistas en evaluación offline, como bugs de integración con features, degradación bajo carga o problemas concentrados en slices raros del tráfico.

9.5 Interleaving experiments

El interleaving es especialmente útil cuando la salida del modelo es un ranking, como una lista de noticias relevantes por activo. En lugar de asignar usuarios completos a A o B, se mezclan resultados de ambos modelos en una misma lista final siguiendo una regla controlada, y se mide interacción (clics, tiempo de lectura). Si los elementos propuestos por B reciben consistentemente más atención, se infiere que su ranking es mejor, y suele requerir menos tráfico que un A/B clásico para detectar diferencias.

La ventaja del interleaving es su sensibilidad a mejoras pequeñas en rankings; su limitación es que añade complejidad de implementación y no aplica cuando la salida es un valor numérico aislado sin ordenamiento.

9.6 (Opcional) Bandits

Si el proyecto escala y se prueban muchas variantes con frecuencia, los métodos de *multi-armed bandits* pueden extender el A/B al asignar tráfico de forma adaptativa, aumentando exposición a la variante que mejor funciona sin dejar de explorar. Para un primer despliegue, suele ser suficiente combinar shadow, A/B y canary, dejando bandits como herramienta posterior cuando el volumen y el ritmo de experimentación lo justifiquen.

9.7 Criterios de éxito y gobernanza del despliegue

Más allá de la técnica, el libro enfatiza definir gobernanza: criterios de éxito, criterios de seguridad y un proceso explícito de aprobación. Los criterios de éxito establecen qué métricas deben mejorar (o al menos no empeorar), en qué magnitud y durante cuánto tiempo para considerarse estable. Los criterios de seguridad determinan umbrales que disparan rollback inmediato, como error, latencia o fallas del servicio. El proceso de aprobación define quién toma la decisión final de promover un modelo y cómo se documentan experimentos y resultados (experiments log, fichas de versión).

En el contexto académico, esto se traduce en demostrar que el modelo nuevo supera baselines (cap. 6), se integra correctamente al pipeline (cap. 7) y respeta las consideraciones de riesgos y monitoreo (cap. 8), dejando evidencia de los experimentos (shadow, A/B, canary, interleaving) como parte de la documentación final del proyecto.

Referencias

Huyen, C. (2022). *Designing Machine Learning Systems: An iterative process for Production-Ready applications*. O'Reilly Media, Inc. Recuperado de:
<https://softouch.on.ca/kb/data/Designing%20Machine%20Learning%20Systems.pdf>.